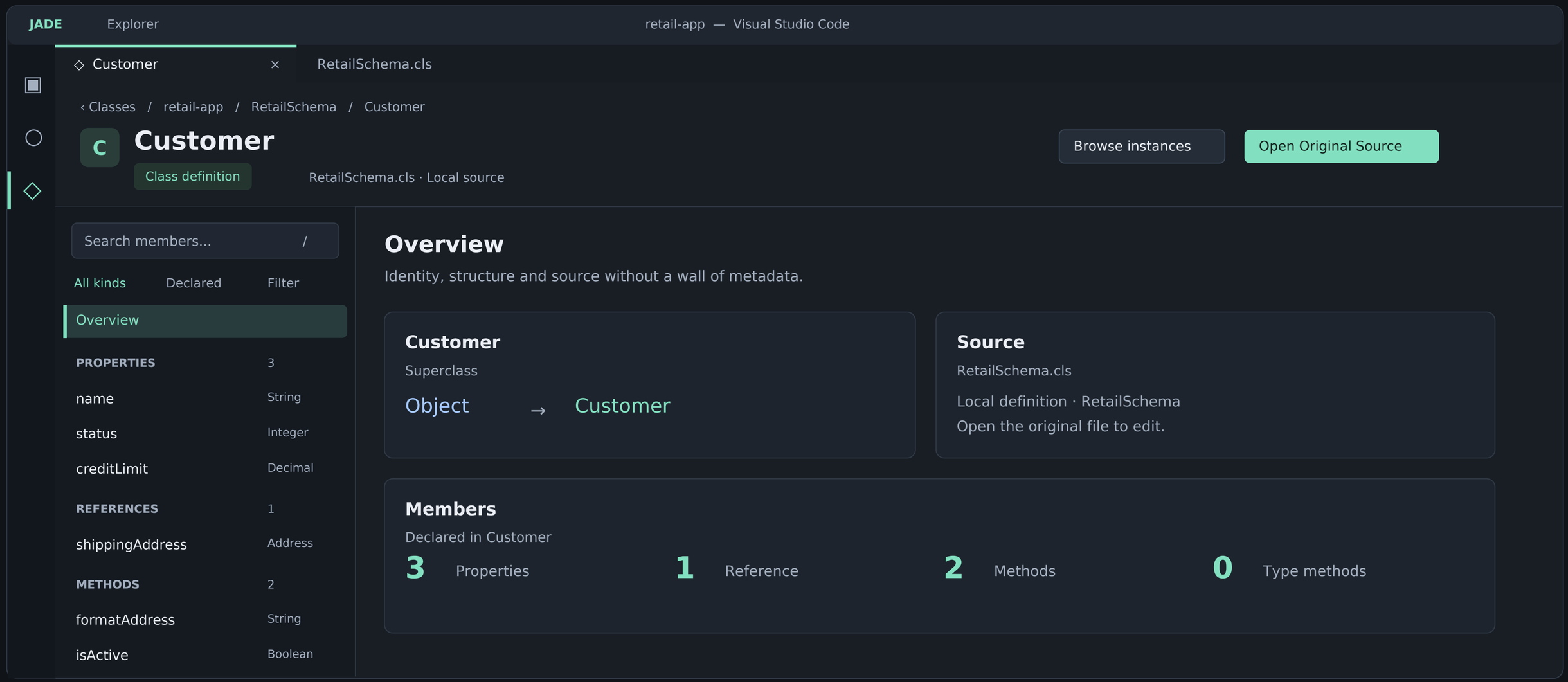
01 / CLASS OVERVIEW

# Understand the class. Then go deeper.

One stable workspace, one member rail, one focused canvas. Familiar to VS Code, distinctively Jade.




**01** Identity stays visible.

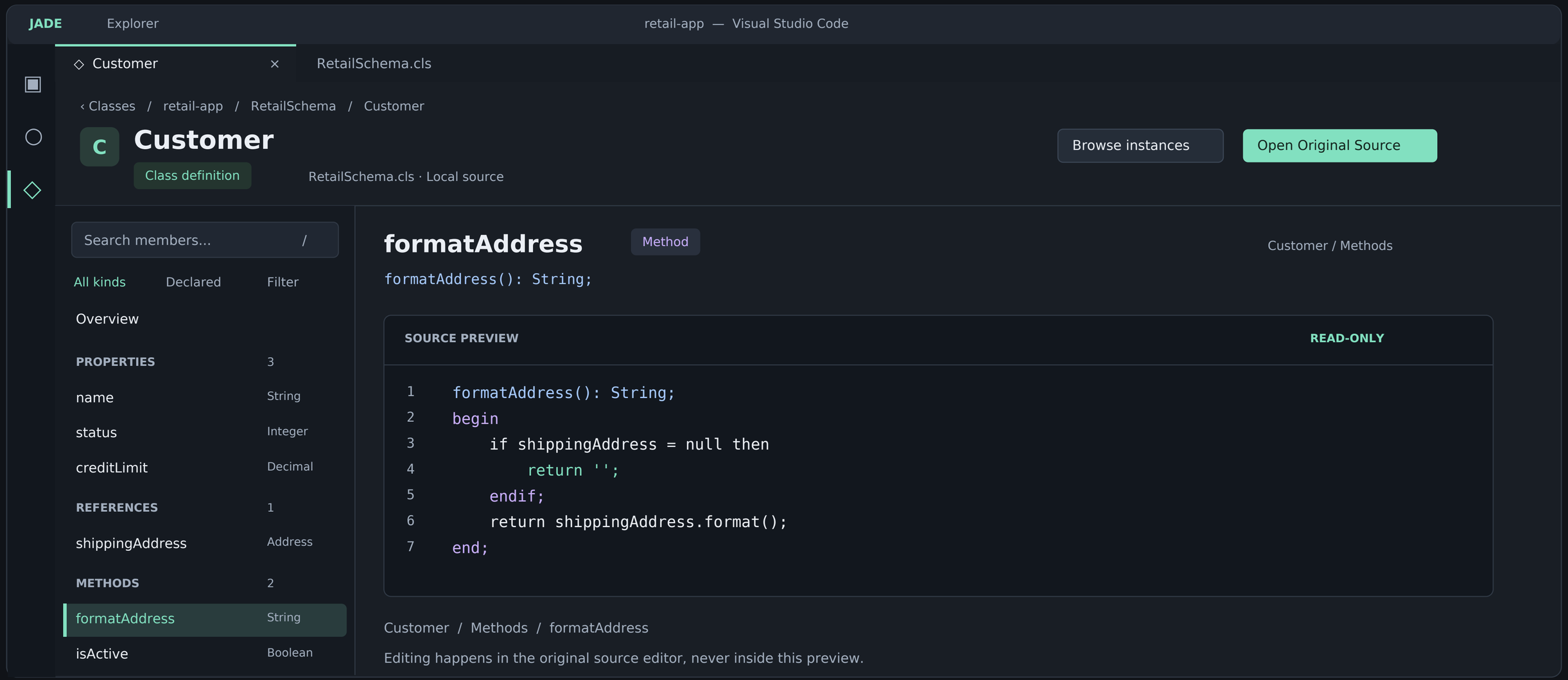
Search covers every member kind.

Details change in place; the class does not disappear.

02 / METHOD SELECTED

# Read the method without losing your place.

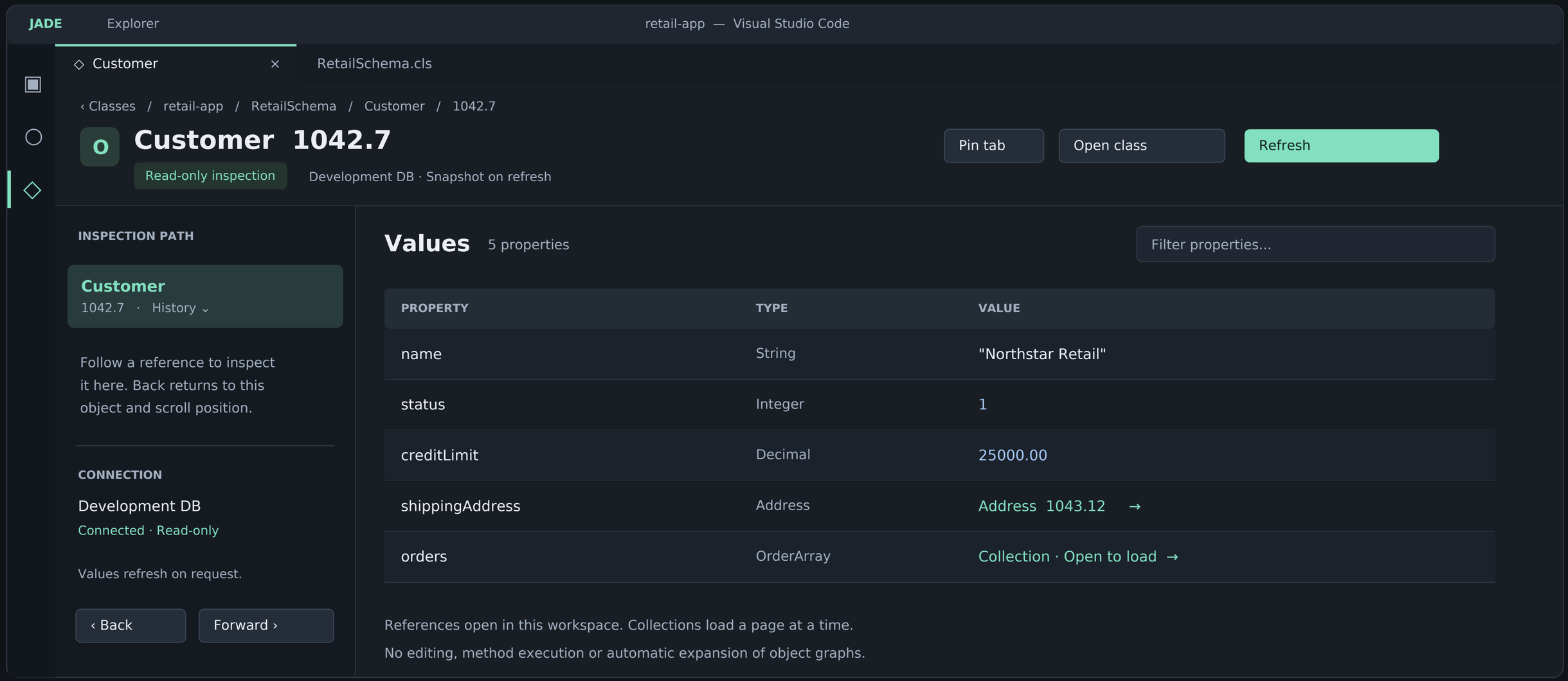
The rail, class identity and member selection remain stable while the source canvas changes.




**02** Single selection updates this panel. Open Original reveals the member in the editable .cls file.

# An object is values and relationships.

Keep a real instance separate from its class definition. Show its connection, identity and read-only state.




**03** A snapshot, not a live watch.

Null, unavailable and not loaded are distinct states.

# Small width. Same mental model.

Below 760 CSS px, replace the fixed rail with an accessible member picker. Never squeeze three columns.

◇ Customer ×

retail-app / RetailSchema

## Customer

Open Original

Class definition

Members / formatAddress ⌄

### formatAddress

`formatAddress(): String;`

```
formatAddress(): String;
begin
  if shippingAddress = null then
    return '';
  endif;
  return shippingAddress.format();
end;
```

Read-only preview

## The member rail becomes a picker

Same search. Same groups. Same selection.

Opening it does not navigate away from the class.

**Members** ×

format

METHODS

formatAddress String

↑ ↓ Navigate Enter Select Esc Close

- At 760 px and above: resizable 240–320 px rail.
- Below 760 px: one canvas and a full-width picker.
- Below 480 px: actions move into a labelled More menu.
- Object rows become property / type / value blocks.
- Source scrolls horizontally; the shell never does.

05 / INTERACTION CONTRACT

# Build one predictable browsing loop.

Selection is navigation. Source editing and database mutations stay outside this workspace.

## Find and select

### Open a class

An existing class-tree action opens or reveals its workspace. Opening another member of that class reuses this panel.

### Search members

Filter by name across all member kinds. Preserve group order and sort each group by name. Show result count and Clear.

### Select a member

Single click or Enter updates the canvas. Property: declaration and type. Reference: declared target type. Method: source. Constants and keys: source + known detail.

### Scope honestly

Default to declared members. Show an Inherited filter only when ownership and inheritance data are actually resolved.

## Navigate and retain

### Identity keys

Class: workspace/project identity + schema + class. Object: connection/session identity + schema + object identity.

### Context preservation

Retain member, search, filters, rail width, canvas scroll and rail scroll per class. Serialize class view state across reloads; reconnect objects explicitly after restart.

### Reference history

Follow a reference in the object workspace. Back restores prior object and scroll. Detect cycles by identity; do not recurse.

### Source synchronization

Track source revisions. Preserve selection by stable identity, not list index. Ignore stale responses after a new selection.

## Keyboard and focus

### Member navigation

Tab reaches search, filters, member tree, canvas and actions in logical order. Use arrow keys for tree navigation; Enter selects.

### Search shortcut

Slash focuses member search only when focus is outside an editable control. Escape clears search; in a picker it closes.

### Return focus

Closing the picker restores its trigger. Selecting a member focuses its detail heading and announces name and kind.

### No surprise editing

Open Original Source reveals the selected range in the normal .cls editor. No inline Save, object mutation or method execution.

# Be precise about what is known.

Never disguise absent metadata as zero, null, a live value or a working action.

## Class and source

### Available foundation

class-view-model.js provides groups, owner, schema, file, member ranges and preview content from the parsed .cls source.

### Grouped members

Support Properties, References, Methods, Type methods, Constants, Collection keys and Inheritance. Hide empty groups.

### Preview loading or failure

Keep header and rail visible. Show a short loading state in the canvas. If unavailable, explain why and offer Retry / Open Original.

### Missing or changed member

If the selected member disappears, return to Overview with a non-blocking notice. Never silently select a different member.

## Objects and collections

### Existing transport

Reuse Object Explorer inspection session, validated identity and read-only transport. Display returned property/type/value data.

### Lazy traversal

Load references only on selection. Show null as null, unavailable with a reason, and unloaded references as navigation targets.

### Paged collections

Use the existing page/cursor mechanism. Show returned items and Next/Previous only when supported. Do not invent totals.

### Disconnect / refresh

Keep last values with a stale banner and disable traversal until reconnected. Refresh replaces the snapshot; never implies watch.

## Resilience and feedback

### No search results

Show “No members match…” and Clear filters. Keep identity and the current selection available outside the results.

### No source / no properties

Use separate empty states: no declared members, source unavailable, and no properties returned. Never a blank panel.

### Connection errors

Show target, plain-language error and Retry or existing connection/configuration command. Do not expose credentials.

### Read safely

Cancel or ignore old requests on class, object or connection change. Treat preview content as text; maintain CSP and escaping.

07 / VISUAL SYSTEM

# A calm, technical surface with a clear hierarchy.

The green accent identifies the active context, not every control. The VS Code theme remains the authority.

## Layout and typography

### Density

8 px spacing rhythm; 16–24 px panel padding. Compact 32 px controls, 36 px member rows, 44–48 px object value rows.

### Hierarchy

24 px class title; 20 px canvas title; 13–14 px UI text; 12 px metadata. Source uses the configured editor monospace font.

### Surfaces

Editor background, a subtly distinct rail, one-pixel borders and restrained 4–8 px corners. No gradients or decorative charts.

### Content priority

Identity → selection → detail → action.
Do not add repeated tab strips, metadata accordions or global search inside this view.

## Theme and accessibility

### Use host tokens

Bind surfaces, text, buttons, input, focus and selection to VS Code CSS variables. Use codicons for native semantic icons.

### Light / high contrast

Theme token mapping, not a forced dark palette. Add visible focus outlines and borders in high contrast. Test both themes.

### Readable meaning

Meet 4.5:1 text contrast and 3:1 essential control boundaries. Pair color with text, icons and accessible state announcements.

### Assistive technology

Use tree semantics for grouped members, table semantics for values, labelled inputs, live result counts and reduced motion.

## Palette direction

### Illustrative dark theme

Canvas #101318 / Surface #191E25
Border #303945 / Text #EBEFF5
Muted #A3AFBF / Accent #82E0C0

### Semantic accents

Green: selection and reference targets.
Blue: types and source identifiers.
Violet: methods and language keywords.

### Scale and zoom

Support 200% zoom and a 360 CSS px editor-group width. No clipped actions. Long names wrap or expose full tooltips.

### Motion

No animated panel switching. A short loading indicator is enough. Keep focus and geometry stable while content arrives.

08 / BUILD BOUNDARY AND ACCEPTANCE

# A concrete, testable first release.

New presentation and navigation over existing source and inspection services. No GitHub changes made.

## Implementation plan

### 1. Class workspace

Replace the separate source-preview presentation with a class-scoped shell. Reuse parsed groups and source previews.

### 2. Safe state model

Add project-aware keys, revision guards, selection and per-class view state. Preserve Open Original Source and native tree entry.

### 3. Object workspace

Apply the same identity and canvas system to existing property/type/value output, reference navigation and paged collections.

### 4. Responsive + accessible

Add compact picker, keyboard navigation, theme tokens, focus return and error states. Do not require database access for class UI.

## Acceptance checklist

### Class behavior

A class opens one workspace per project. Selecting any member preserves search, rail position and class context.

### Source behavior

Preview remains read-only; Open Original reveals the correct member. Switching quickly never shows the previous member response.

### Object behavior

Target and identity are always visible. Reference Back restores context. Collections page without eagerly loading all objects.

### Quality gate

Test duplicate schema/class across projects, deleted member, lost connection, null values, large member lists, 360 px width, keyboard, 200% zoom, light/dark and high contrast.

## Scope and traceability

### Explicitly out of scope

Inline source editing, object updates, live watch, method execution, a new schema browser, impact graphs and new DB APIs.

### Conditional metadata

Counts and inheritance are displayed only from resolved data. Missing information is labelled unavailable rather than inferred.

### Grounded in existing modules

extension/src/class-view-model.js
extension/src/source-member-preview.js
extension/src/object-explorer.js
extension/media/object-page.js

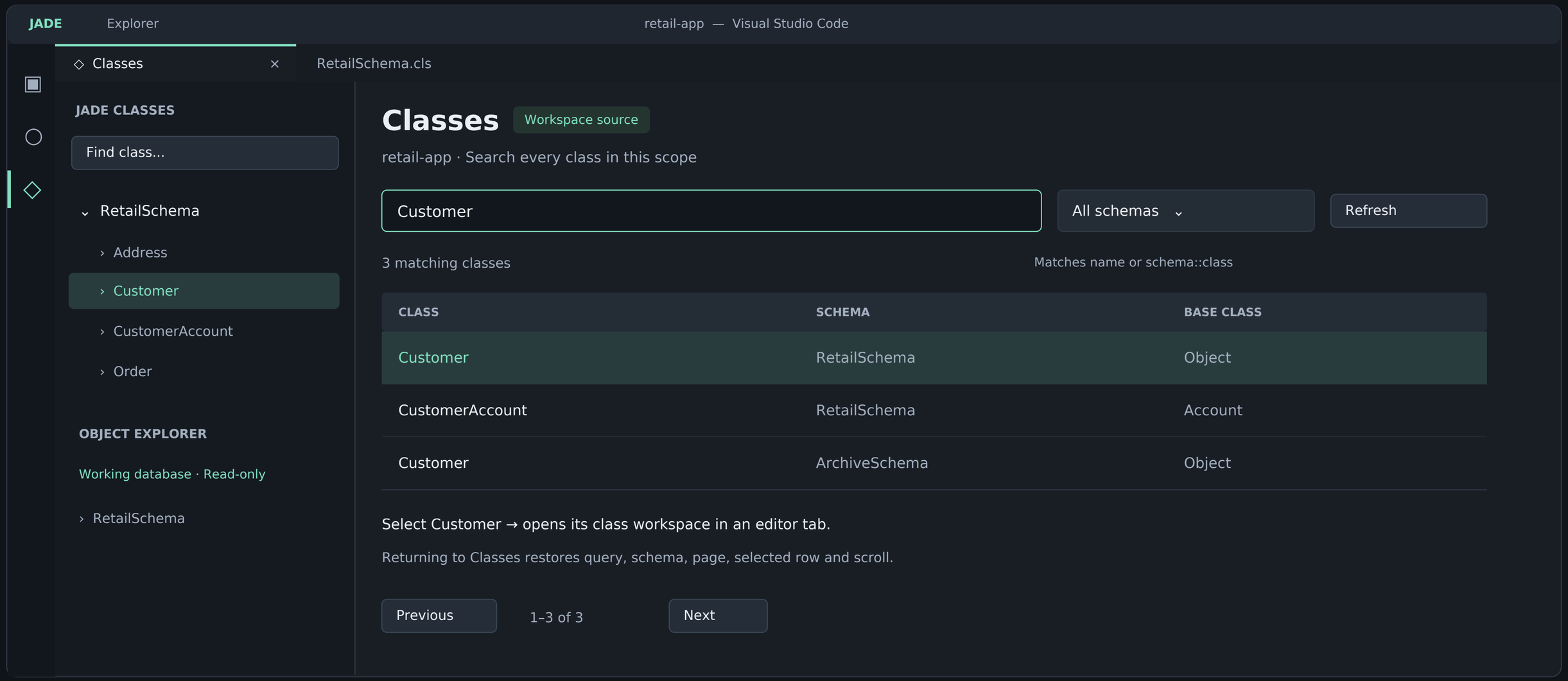
### Design assumptions

Customer and shown values are illustrative. The proposed UX is not a report of current functionality. Target: GitHub issue #30.

09 / EXISTING DISCOVERY → NEW WORKSPACE

# Keep the class browser. Connect the journey.

Evolve the existing catalogue and QuickPick, rather than adding a competing class search inside the workspace.




09 Source catalogue → Class → Member

Database catalogue → Instances → Object → Reference → Back

# Make every route lead somewhere useful.

Two honest scopes: workspace definitions and database instances. Link them explicitly, never silently conflate them.

## Discover → class

### Reuse both existing entry points

Keep native Find Class QuickPick and the paged catalogue. Use their shared matcher for class name and schema::class queries.

### Catalogue state

Keep query, schema and offset. Add selected class and scroll restoration. Query/schema changes reset offset; Back restores it.

### Open behavior

Workspace result opens or reveals the class workspace. Tree member opens that class with the member already selected.

### Scale and keyboard

Retain 100-row catalogue pages and full metadata search. Enter opens focused row; Tab reaches filters and paging. QuickPick keeps its native keyboard behavior.

## Class → instances → object

### Explicit context bridge

Browse instances uses the selected or explicitly chosen inspection target. Show working/source DB, schema and class.

### Keep the existing instance path

Reveal the matching class in Object Explorer and use its existing paged instance list. Selecting an instance opens the object view.

### Database catalogue behavior

Keep its current class reveal/instance flow. Do not route a database result to local source as if local definitions were database truth.

### Object → source

Open class source resolves schema + class in the project. If missing, explain that source must be pulled. Do not pull automatically.

## Return + verify

### No dead ends

Classes breadcrumb returns to origin. Keep source and database browse state separate by project or connection scope.

### Preserve existing affordances

Retain object Back/Forward, history and Pin tab. Compact layouts place History and Pin in an accessible labelled More menu.

### Acceptance scenarios

Search Customer → choose RetailSchema → select method → return: results unchanged. Browse instances → inspect → follow ref → Back: original object and scroll restored.

### Grounding and limits

class-browser-ui.js, class-catalogue.js and class-catalogue.js UI already support these discovery paths. New work is the workspace, context bridge and richer return-state model.